

IN1160, uke 3:

Klassifikasjon

Erik Velldal

Språkteknologigruppen (LTG)
Seksjon for maskinlæring
Institutt for informatikk
Universitetet i Oslo

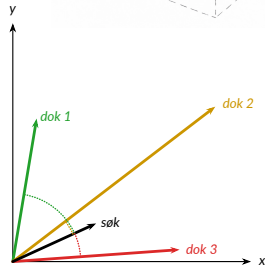
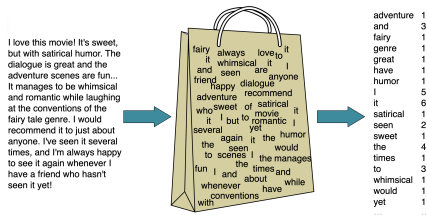
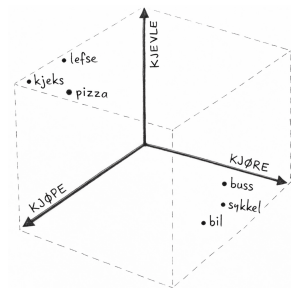
4. februar, 2026

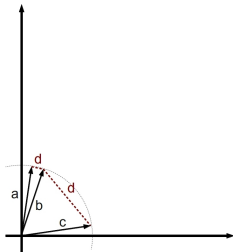
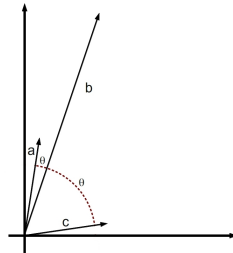
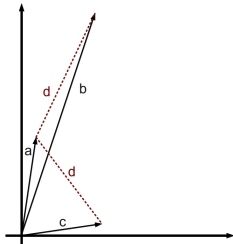


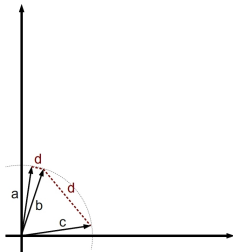
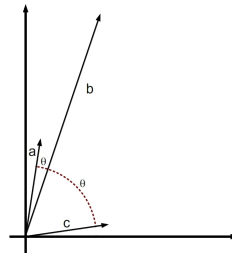
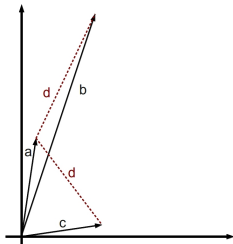
Forrige uke



- Trekk og **vektorrommodeller** for å representere data numerisk
- Avstandsmål og naborelasjoner
- Representasjon av tekst basert på 'ordsekker' (*bag-of-words*)
- Ordvektorer og dokumentvektorer







- Euklidsk avstand mellom punkter.
- Kosinus-likhet av vektorvinkler.
- Rå tellinger vektet av TF-IDF.
- Kan unngå lengdesensitivitet med normalisering.

Neste tema: to kategoriseringsproblemer i ML



Klyngeanalyse

- Ikke-veiledet læring fra uannoterte data
- Automatisk gruppering av like objekter
- Ingen forhåndsdefinerte klasser eller merkelapper
- Tema for neste uke

Neste tema: to kategoriseringsproblemer i ML



Klyngeanalyse

- Ikke-veiledet læring fra uannoterte data
- Automatisk gruppering av like objekter
- Ingen forhåndsdefinerte klasser eller merkelapper
- Tema for neste uke

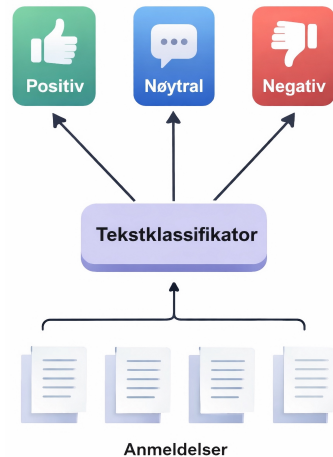
Klassifikasjon

- Veiledet læring fra annoterte treningsdata
- Basert på et sett forhåndsdefinerte klasser og treningseksempler.
- Skal se på to eksempler på klassifikasjon knyttet til vektorrom denne uka:
 - Rocchio
 - k-Næreste Naboer (kNN)
 - Kommer tilbake til klassifikasjon senere, f.eks. beslutningstrær og logistisk regresjon.
- Skal også dekke evaluering

Hva inngår i klassifikasjon?



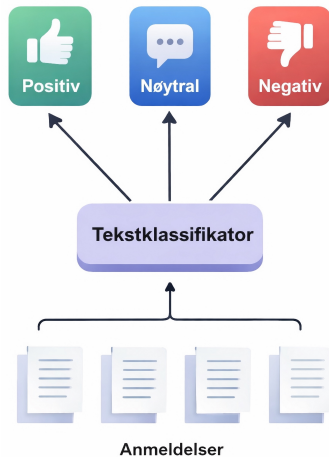
- Et **domene** av objekter/observasjoner som skal klassifiseres.



Hva inngår i klassifikasjon?



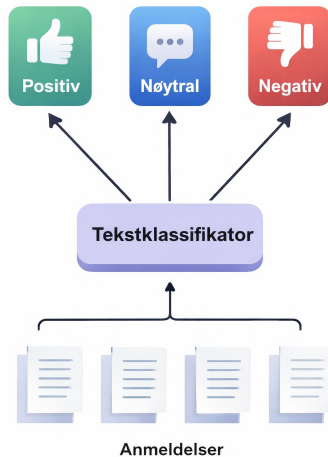
- Et **domene** av objekter/observasjoner som skal klassifiseres.
- Et endelig sett med **merkelapper**; navngitte klasser.



Hva inngår i klassifikasjon?



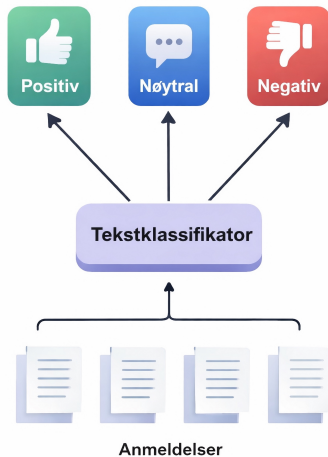
- Et **domene** av objekter/observasjoner som skal klassifiseres.
- Et endelig sett med **merkelapper**; navngitte klasser.
- **Annoterte data**: Forhåndsdefinerte eksempler på par av objekter + riktig merkelapp;
- trengs for både **trening** og **evaluering**.
- Annotasjonene utgjør 'veiledningen'.



Hva inngår i klassifikasjon?



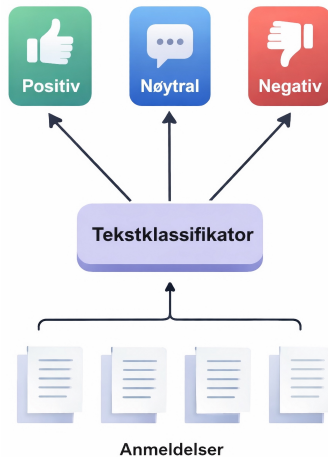
- Et **domene** av objekter/observasjoner som skal klassifiseres.
- Et endelig sett med **merkelapper**; navngitte klasser.
- **Annoterte data**: Forhåndsdefinerte eksempler på par av objekter + riktig merkelapp;
- trengs for både **trening** og **evaluering**.
- Annotasjonene utgjør 'veiledningen'.
- **Klassifikator**: modell trent for å tilordne et gitt objekt en merkelapp.



Hva inngår i klassifikasjon?



- Et **domene** av objekter/observasjoner som skal klassifiseres.
- Et endelig sett med **merkelapper**; navngitte klasser.
- **Annoterte data**: Forhåndsdefinerte eksempler på par av objekter + riktig merkelapp;
- trengs for både **trening** og **evaluering**.
- Annotasjonene utgjør 'veiledningen'.
- **Klassifikator**: modell trent for å tilordne et gitt objekt en merkelapp.
- Utdata fra klassifikatoren omtales ofte som **prediksjoner**.



Eksempler på klassifikasjon i NLP



Tekstklassifikasjon

Tekstklassifikasjon

- Sentimentanalyse
- Tematikk
- Forfatteridentifikasjon
- Sjøppelpost (*spam*)
- Språkidentifikasjon
- Hatprat
- Osv.

Tekstklassifikasjon

- Sentimentanalyse
- Tematikk
- Forfatteridentifikasjon
- Sjøppelpost (*spam*)
- Språkidentifikasjon
- Hatprat
- Osv.

Andre klassifikasjonsoppgaver i NLP

Tekstklassifikasjon

- Sentimentanalyse
- Tematikk
- Forfatteridentifikasjon
- Sjøppelpost (*spam*)
- Språkidentifikasjon
- Hatprat
- Osv.

Andre klassifikasjonsoppgaver i NLP

- Ordklassetaggning (*Part-of-speech tagging*)
- Navnegjenkjenning (*NER: Named entity recognition*)
- Hendelsesanalyse (*Event extraction*)
- Språkmodeller
 - Predikerer neste ord i en sekvens.
- Osv.

Tekstklassifikasjon

- Sentimentanalyse
- Tematikk
- Forfatteridentifikasjon
- Søppelpost (*spam*)
- Språkidentifikasjon
- Hatprat
- Osv.

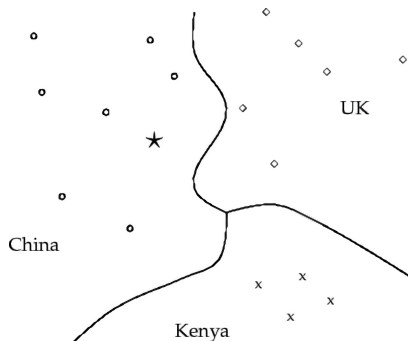
Andre klassifikasjonsoppgaver i NLP

- Ordklassetaggning (*Part-of-speech tagging*)
- Navnegjenkjenning (*NER: Named entity recognition*)
- Hendelsesanalyse (*Event extraction*)
- Språkmodeller
 - Predikerer neste ord i en sekvens.
- Osv.

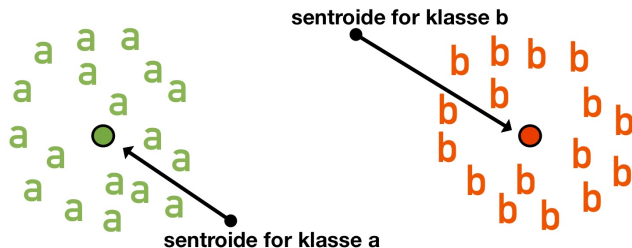
- Fokus i **oblig 1b**: **temaklassifikasjon** av avisartikler med **kNN** og **BoW**-trekk.

- **Binær**: to klasser
- **Multiklasse** (*multi-class*): tre eller flere gjensidig utelukkende klasser.
- Noen læringsalgoritmer er i utgangspunktet binære (f.eks. perceptron, logistisk regresjon) og må tilpasses multiklasse-problemer; f.eks. med én modell for hver klasse.
- Både Rocchio og kNN håndterer multiklasse-problemer naturlig.
- **Multietikett** (*multi-label*): hvert objekt kan tilhøre flere klasser samtidig. Igjen løses det ofte med separate modeller for hver klasse.

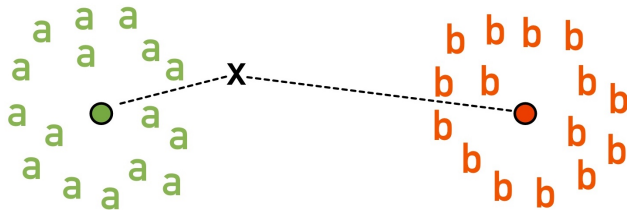
- I en vektorromsmodell representeres objekter som punkter.
- **Klasser** er samlinger av punkter.
- Klassifikasjon i vektorrom antar det M,R&S kaller '**kontiguitetshypotesen**':
- Objekter i samme klasse danner sammenhengende **regioner**, uten overlapp med andre klasser.
- Klassifikasjon går ut på å beregne – implisitt eller eksplisitt – grenser som skiller klassene fra hverandre; de såkalte **beslutningsgrensene** (*decision boundaries*).



- Hver klasse C_i representeres av sin **sentroide** (*centroid*), μ_i .
- Kan tenkes på som et 'tyngdepunkt', midtpunkt eller prototype.
- Beregnes ved gjennomsnittet av vektorene $\mathbf{x}_j$ til medlemmene; $\mu_i = \frac{1}{|C_i|} \sum_{\mathbf{x}_j \in C_i} \mathbf{x}_j$



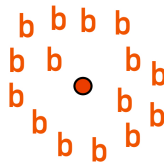
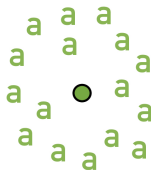
- **Beslutningsregelen** i Rocchio-klassifikasjon:
- For å klassifisere et nytt objekt o_j , representert ved en trekkvektor x_j :
 - avgjør hvilken sentroide μ_i som er nærmest x_j , f.eks. ved euklidsk avstand,
 - og tilordne objektet den tilsvarende klassen i .
- Kan sees på som en **nærmeste-sentroide-klassifikator**.



Rocchio er et eksempel på en lineær klassifikator



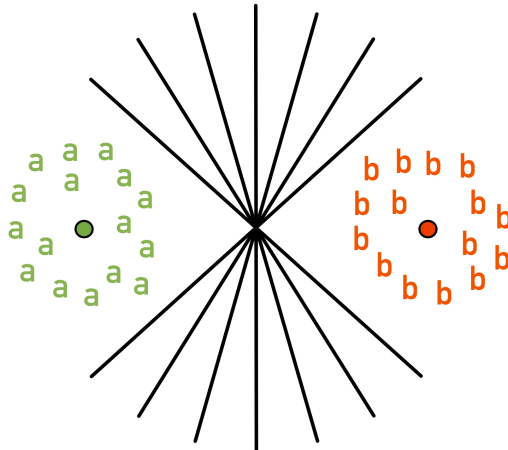
- I to dimensjoner tilsvarer **beslutningsgrensen** som skiller klassene en **rett linje**.



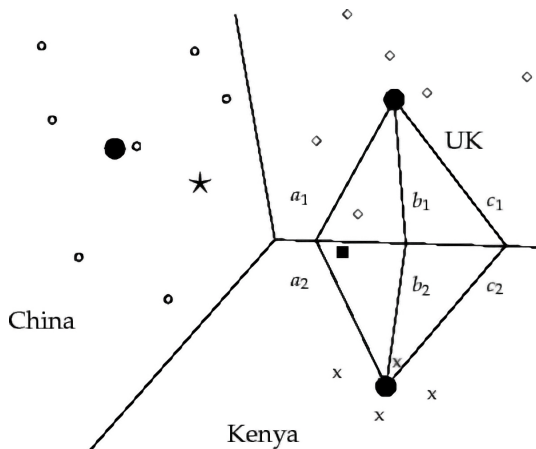
Lineært separerbare klasser



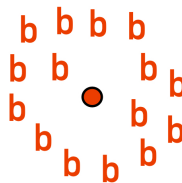
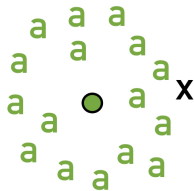
- Men det finnes uendelig mange måter å definere beslutningsgrensen på.



- Grensen mellom to klasser definert ved **mengden av punkter som er like langt unna sentroidene**.
- I to dimensjoner danner disse punktene en **linje**.
- Danner et plan i 3D eller et **hyperplan** ved høyere dimensjonalitet.
- Grensene *impliseres* av beslutningsregelen; beregnes ikke eksplisitt!



- Klassifikasjonen er kun basert på avstanden til sentroiden;
- ignorerer den lokale fordelingen av medlemmer innen klassene.
- Antar implisitt at klassene er kuler med tilnærmet samme radius.
- Lite egnet for klasser som ikke kan representeres presist av én enkelt prototype eller ett sentrum (f.eks. usammenhengende eller avlange regioner).
- Siden Rocchio gir en lineær beslutningsgrense, egner metoden seg kun for problemer med lineært separerbare klasser.



Problematisk: Avlange regioner

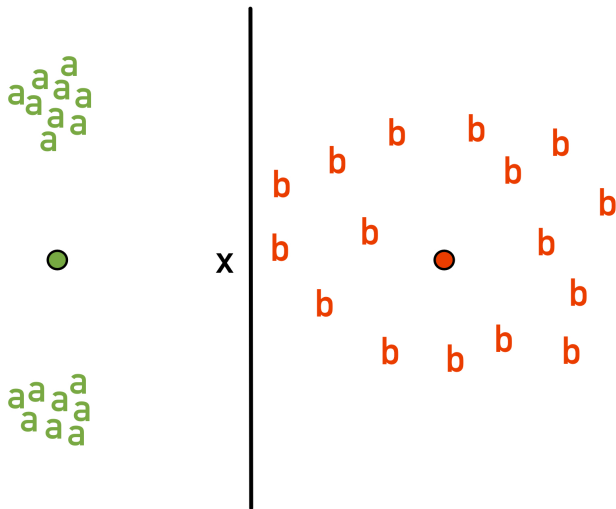


a
a
a
a
●
a
a
a
a

x

b b b b ● b b b

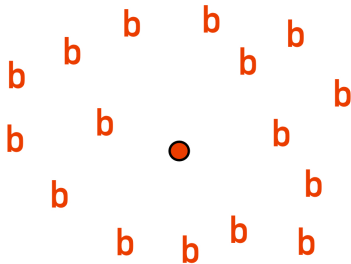
Problematisk: Ikke-sammenhengende regioner



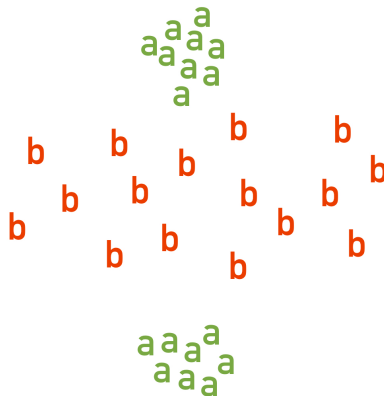
Problematisk: Ulike størrelser



x



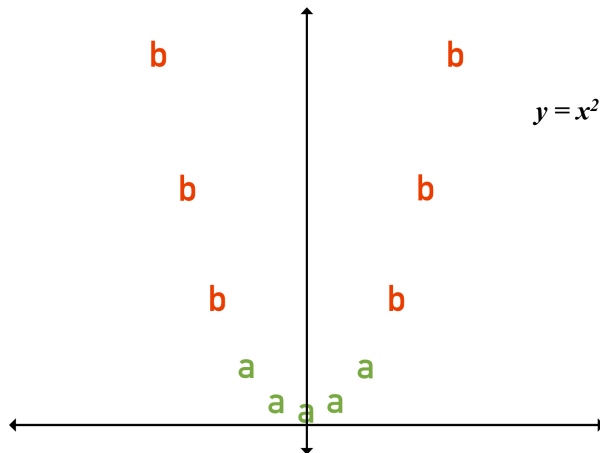
Problematisk: Ikke-lineær klassegrænse





- Før vi går videre til ikke-lineære klassifikatorer, merk at: Klasser som ikke er lineært separerbare i et gitt trekkrom...

A side-note on non-linearity



- ...kan bli lineært separerbare når trekkene projiseres til høyere dimensjonalitet (dette er grunnlaget for såkalte *kernel*-metoder).



Beslutningsregel

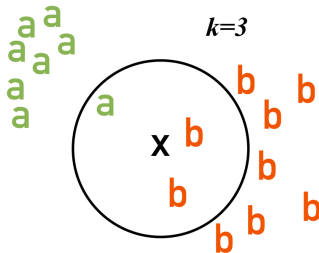
- Parameteren k må spesifiseres på forhånd.
- Ranger treningseksemlene etter avstand til testobjektet x .
- For $k = 1$: x tilordnes samme klasse som sin nærmeste nabo.
- For $k > 1$: x tilordnes majoritetsklassen blant sine k nærmeste naboer.

k NN: k Nærmeste Nabo-klassifikasjon (k -Nearest Neighbors)



Beslutningsregel

- Parameteren k må spesifiseres på forhånd.
- Ranger treningseksemplene etter avstand til testobjektet x .
- For $k = 1$: x tilordnes samme klasse som sin nærmeste nabo.
- For $k > 1$: x tilordnes majoritetsklassen blant sine k nærmeste naboer.

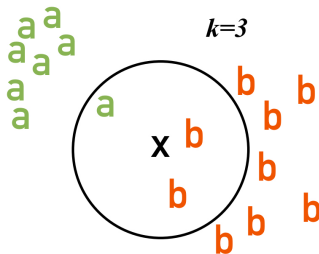


k NN: k Nærmeste Nabo-klassifikasjon (k -Nearest Neighbors)



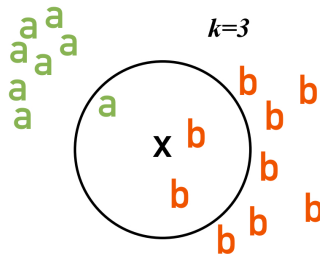
Beslutningsregel

- Parameteren k må spesifiseres på forhånd.
- Ranger treningseksemlene etter avstand til testobjektet x .
- For $k = 1$: x tilordnes samme klasse som sin nærmeste nabo.
- For $k > 1$: x tilordnes majoritetsklassen blant sine k nærmeste naboer.



Beslutningsregel

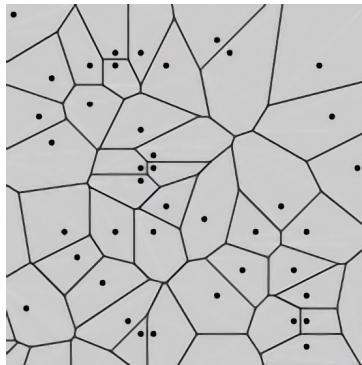
- Parameteren k må spesifiseres på forhånd.
- **Ranger** treningseksemlene etter avstand til testobjektet x .
- For $k = 1$: x tilordnes samme klasse som sin nærmeste nabo.
- For $k > 1$: x tilordnes majoritetsklassen blant sine k nærmeste naboer.



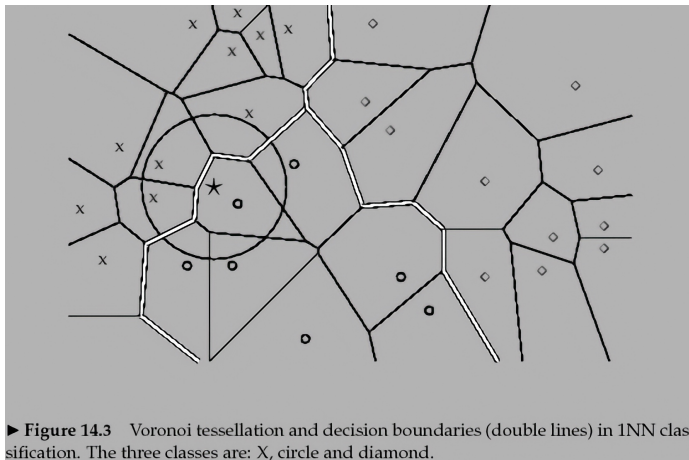
Kommentarer

- Eksempel på en **ikke-lineær** klassifikator.
- I motsetning til Rocchio bestemmes k NN-**beslutningsgrensen** lokalt.
- Defineres av den såkalte **Voronoi-tesselleringen**.

- Anta $k = 1$:
- For et gitt sett av objekter i rommet;
- la hvert objekt definere en celle bestående av alle punkter som er nærmere dette objektet enn noe annet objekt.
- Resulterer i et sett av konvekse polygoner; såkalte **Voronoi-celler**.
- For $k \geq 1$ er Voronoi-cellene gitt ved regionene i rommet der mengden av de k nærmeste naboene er den samme.



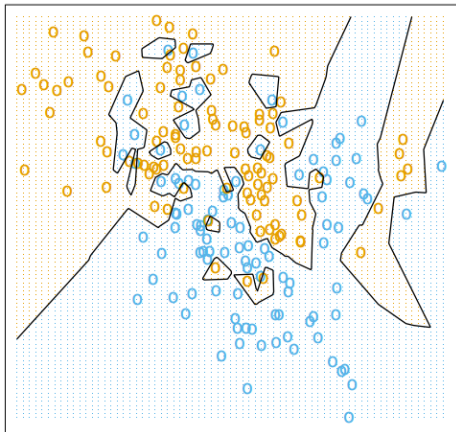
Voronoi-tesselling for 1NN



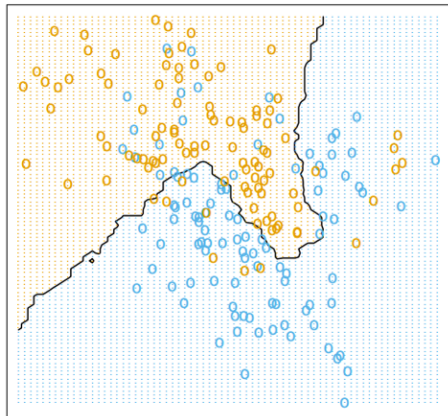
► **Figure 14.3** Voronoi tessellation and decision boundaries (double lines) in 1NN classification. The three classes are: X, circle and diamond.

Beslutningsgrense for 1NN: definert langs kantene til Voronoi-cellene for treningseksempelene i hver klasse. Viser **ikke-lineariteten** til kNN.

1-Nearest Neighbor Classifier



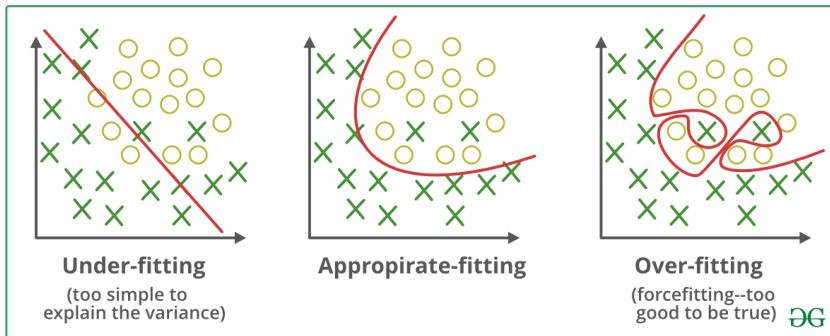
15-Nearest Neighbor Classifier



Bilde fra Elements of Statistical Learning, Hastie et al. 2009

- Hva skjer om vi angir $k = N$?

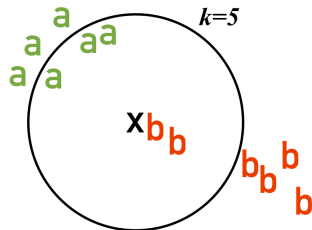
- **Overtilpasning** (*overfitting*): modellen tilpasses treningsdataene svært godt, men generaliserer dårlig, dvs. gir mange feil på testdata.
- **Undertilpasning** (*underfitting*): modellen gjør mange feil selv på treningsdataene, dvs. den er ikke i stand til å beskrive dataene.



- **Bias-variance**-avveiningen refererer til balansering av to feilkilder:
- **Bias**: effekten av antakelser i modellen. For enkle modeller har høy 'bias', noe som fører til **undertilpasning**.
- **Varians**: følsomhet for endringer i treningsdataene. Svært komplekse modeller kan ha høy varians, noe som fører til **overtilpasning**.
- Eksempel for kNN:
 - $k = 1$: svært høy varians, svært lav bias.
 - $k = N$: svært lav varians, svært høy bias.

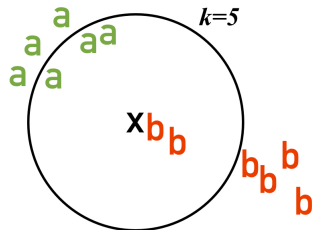
En probabilistisk versjon

- Sannsynligheten for medlemskap i en klasse c gitt ved andelen av de k nærmeste naboene som tilhører c .



En probabilistisk versjon

- Sannsynligheten for medlemskap i en klasse c gitt ved andelen av de k nærmeste naboene som tilhører c .

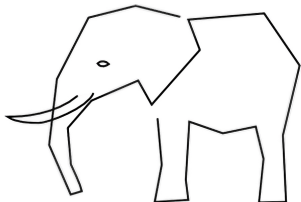
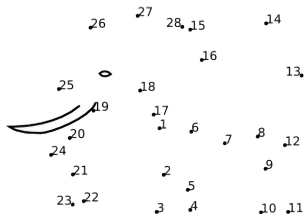


Avstandsvektet versjon

- Poeng for en klasse c_i kan beregnes som: $\text{score}(c_i, o_j) = \sum_{\mathbf{x}_n \in \text{knn}(\mathbf{x}_j)} I(c_i, \mathbf{x}_n) \text{sim}(\mathbf{x}_n, \mathbf{x}_j)$
- der $\text{knn}(\mathbf{x}_j)$ er mengden k nærmeste naboer for $\mathbf{x}_j$, sim er likhetsmålet, og $I(c_i, \mathbf{x}_n)$ er 1 hvis $\mathbf{x}_n \in c_i$ og 0 ellers.
- Kan gi mer nøyaktige resultater og unngå tilfeller av 'uavgjort'.

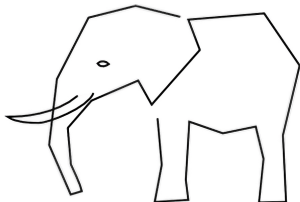
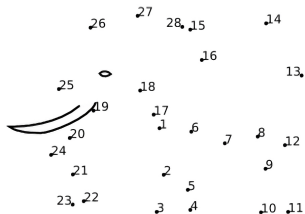
- Involverer egentlig ingen reell *læring*;
- **memorerer** ganske enkelt alle treningseksemplene.
- Eksempel såkalt **minne-basert** (*memory-based*) eller **instans-basert** (*instance-based*) læring.
- Generelt i ML gjelder: jo mer treningsdata, jo bedre.
- Men for k NN kan store treningssett gi en **effektivitetskostnad**.
- **Testtid** er lineær i størrelsen på treningssettet,
- men uavhengig av antall klasser.
- En mulig fordel i oppgaver som involverer mange klasser.
- Legg merke til likheten med problemet *ad hoc informasjonsgjenfinning* (*retrieval*), f.eks. å returnere relevante dokumenter for et gitt søk;
 - Begge handler om å identifisere nærmeste naboer.

Connecting the dots...



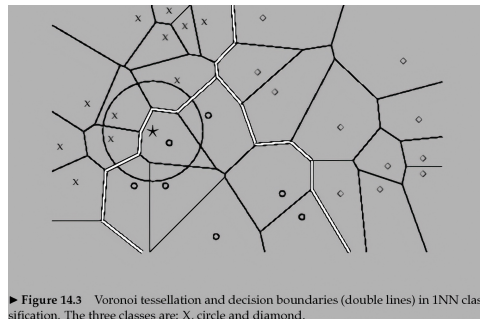
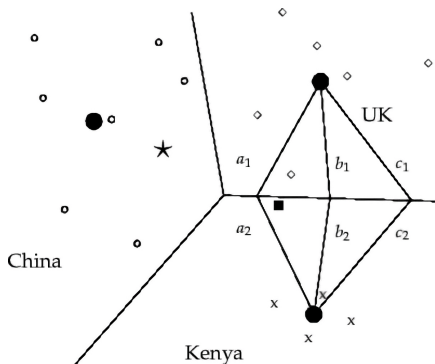
- Kan tenke på **Rocchio** som **1-nærmeste-nabo**-klassifikasjon med hensyn til sentroidene.

Connecting the dots...



- Kan tenke på **Rocchio** som **1-nærmeste-nabo**-klassifikasjon med hensyn til sentroidene.
- Hvordan kan dette ha seg? Er ikke kNN **ikke-lineær** og Rocchio **lineær**?

- Husk at beslutningsgrensen til **kNN** er **lokalt lineær** for hver celle i Voronoi-diagrammet.
- For **Rocchio** bruker vi kun ett eksempel per klasse, nemlig sentroiden, noe som gir **én enkelt Voronoi-celle for hver klasse**.



Neste tema:

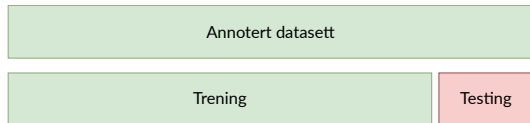


EVALUERING

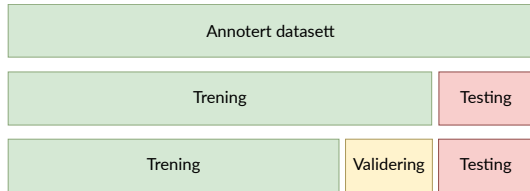
- For å **evaluere** en klassifikator måler vi hvor mange korrekte prediksjoner den gjør for usette testeksempler.
- De annoterte testdataene kalles gjerne for **gullstandarden**, og vi evaluerer ved å sammenligne modellens prediksjoner mot denne.
- Vi skal se på ulike **evalueringsmål**,
- samt ulike **datasplitter**.
- (Hvorfor kan vi ikke teste på treningsdataene?)

Annotert datasett

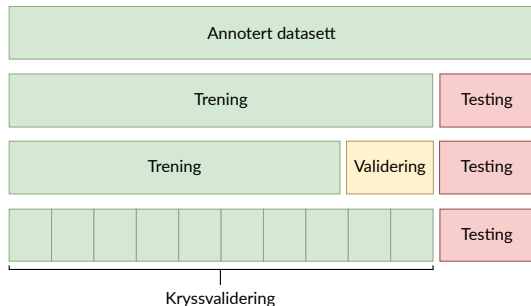
- Vi kan ikke teste på de samme dataene vi trener på.



- Vi kan ikke teste på de samme dataene vi trener på.
- Skiller mellom **treningssettet** og et reservert **testsett** (*held-out test set*).



- Vi kan ikke teste på de samme dataene vi trener på.
- Skiller mellom **treningssettet** og et reservert **testsett** (*held-out test set*).
- Trenger ofte å gjøre gjentatt testing under utvikling av modellen; valg hyperparametre, trekk, osv.
- Skiller ut et eget **valideringssett** for dette (*development / validation set*)

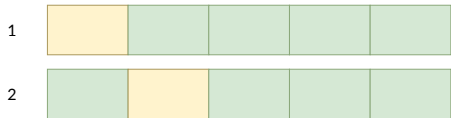


- Vi kan ikke teste på de samme dataene vi trener på.
- Skiller mellom **treningssettet** og et reservert **testsett** (*held-out test set*).
- Trenger ofte å gjøre gjentatt testing under utvikling av modellen; valg hyperparametre, trekk, osv.
- Skiller ut et eget **valideringssett** for dette (*development / validation set*)
- Eller, hvis vi har lite data, ved hjelp av **k-fold kryssvalidering**.

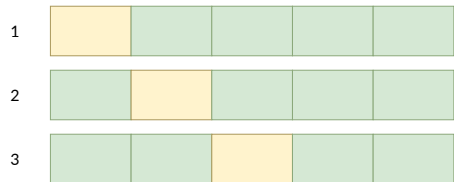
1



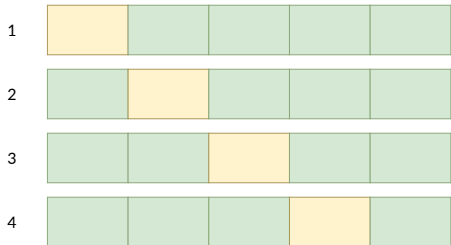
- Eksempel med 5-fold kryssvalidering
- Trener 5 modeller på 5 ulike $\frac{4}{5}$ delmengder av treningssettet.
- For hver gang tester vi på den gjenstående $\frac{1}{5}$ -delen.
- Gjennomsnittet gir oss valideringsresultatet.
- Mest ekstreme tilfelle; 'utelat-én'-kryssvalidering (*leave-one-out*).



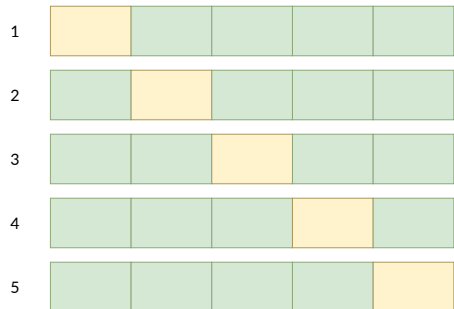
- Eksempel med 5-fold kryssvalidering
- Trener 5 modeller på 5 ulike $\frac{4}{5}$ delmengder av treningssettet.
- For hver gang tester vi på den gjenstående $\frac{1}{5}$ -delen.
- Gjennomsnittet gir oss valideringsresultatet.
- Mest ekstreme tilfelle; 'utelat-én'-kryssvalidering (*leave-one-out*).



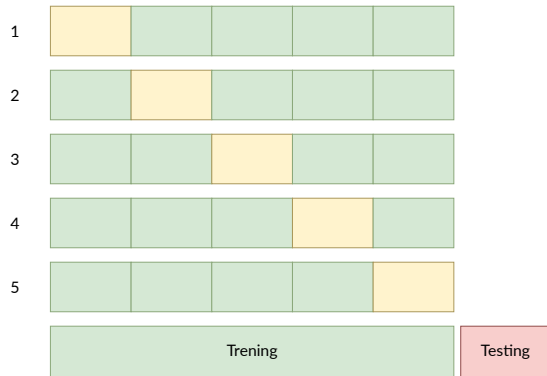
- Eksempel med 5-fold kryssvalidering
- Trener 5 modeller på 5 ulike $\frac{4}{5}$ delmengder av treningssettet.
- For hver gang tester vi på den gjenstående $\frac{1}{5}$ -delen.
- Gjennomsnittet gir oss valideringsresultatet.
- Mest ekstreme tilfelle; 'utelat-én'-kryssvalidering (*leave-one-out*).



- Eksempel med 5-fold kryssvalidering
- Trener 5 modeller på 5 ulike $\frac{4}{5}$ delmengder av treningssettet.
- For hver gang tester vi på den gjenstående $\frac{1}{5}$ -delen.
- Gjennomsnittet gir oss valideringsresultatet.
- Mest ekstreme tilfelle; 'utelat-én'-kryssvalidering (*leave-one-out*).



- Eksempel med 5-fold kryssvalidering
- Trener 5 modeller på 5 ulike $\frac{4}{5}$ delmengder av treningssettet.
- For hver gang tester vi på den gjenstående $\frac{1}{5}$ -delen.
- Gjennomsnittet gir oss valideringsresultatet.
- Mest ekstreme tilfelle; 'utelat-én'-kryssvalidering (*leave-one-out*).



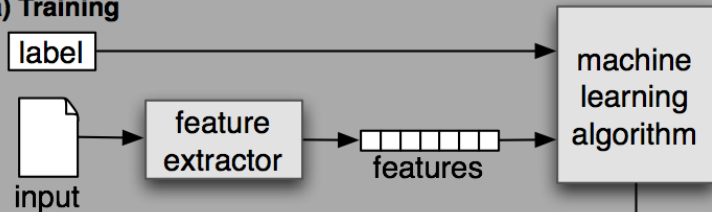
- Eksempel med 5-fold kryssvalidering
- Trener 5 modeller på 5 ulike $\frac{4}{5}$ delmengder av treningssettet.
- For hver gang tester vi på den gjenstående $\frac{1}{5}$ -delen.
- Gjennomsnittet gir oss valideringsresultatet.
- Mest ekstreme tilfelle; 'utelat-én'-kryssvalidering (*leave-one-out*).
- Til slutt er det likevel ønskelig å teste på et reservert **testsett** (kan da trene modellen på hele treningssettet).



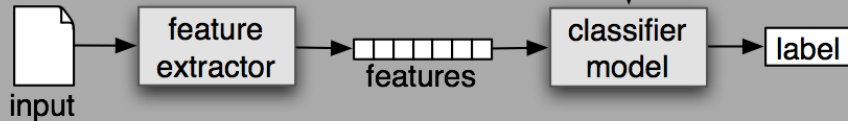
- Før vi trener og anvender en klassifikator, må vi først lage **trekkvektorene** som skal representere dataene våre.
- Omtales også ofte som **vektorisering**.
- **Trekktypene** (f.eks. BoW-vokabularet) må defineres relativt til **treningssettet** (inkludert parametere som frekvens-terskler, idf-vektorer, osv.).
- Når vi **vektorerer testdata**, må vi bruke de samme trekkene som under trening.

- Før vi trener og anvender en klassifikator, må vi først lage **trekkvektorene** som skal representere dataene våre.
- Omtales også ofte som **vektorisering**.
- **Trekktypene** (f.eks. BoW-vokabularet) må defineres relativt til **treningssettet** (inkludert parametere som frekvens-terskler, idf-vektorer, osv.).
- Når vi **vektorerer testdata**, må vi bruke de samme trekkene som under trening.
- Vektorisering gjøres derfor ofte i **to trinn**:
 1. Først defineres trekksettet basert på treningsdataene,
 2. Deretter opprettes selve trekkvektorene (basert på trekkene fra trinn 1.)
 - Trinn 1. gjøres kun for treningssettet; trinn 2. gjøres for hvert datasett / splitt vi ønsker å bruke.
 - Kalles *fit* (1.) og *transform* (2.) i scikit-learn-terminologi.

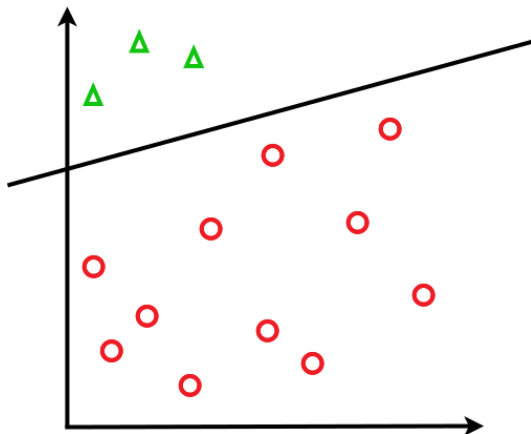
(a) Training

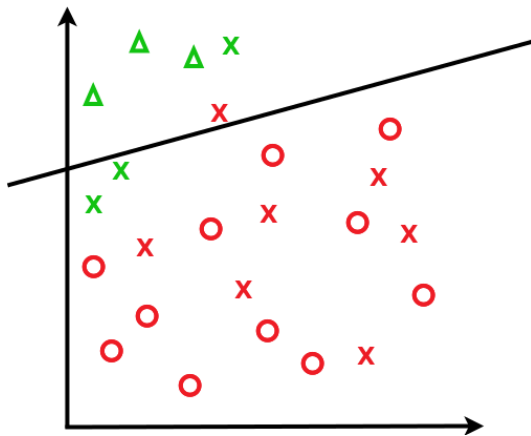


(b) Prediction

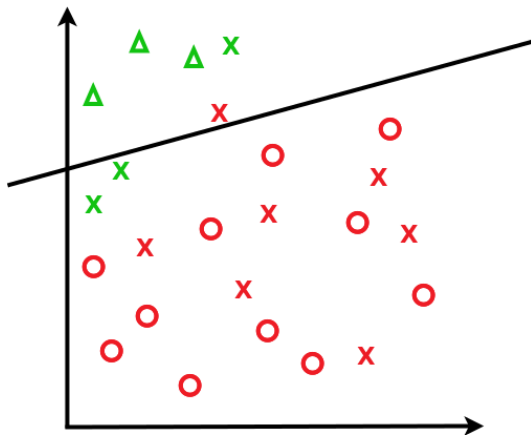


(Image: <https://www.nltk.org/book/ch06.html>)





Riktig og galt på ulike måter: feilmatriser



prediksjon = positiv
prediksjon = negativ

gull = positiv

sann positiv (TP)

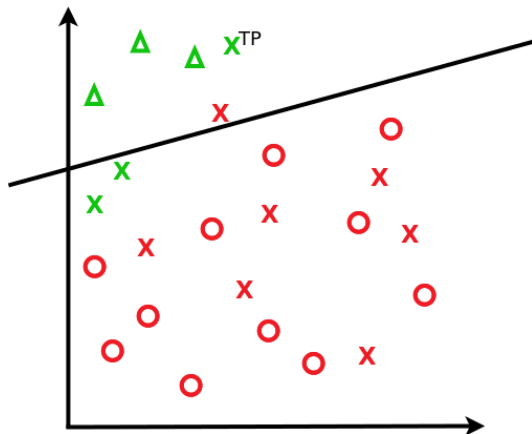
falsk negativ (FN)

gull = negativ

falsk positiv (FP)

sann negativ (TN)

Riktig og galt på ulike måter: feilmatriser



prediksjon = positiv
prediksjon = negativ

gull = positiv

sann positiv (TP)

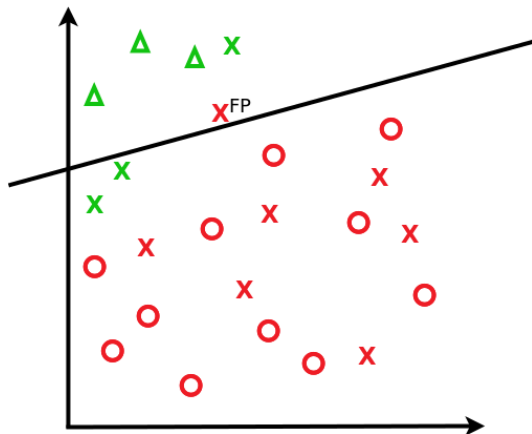
falsk negativ (FN)

gull = negativ

falsk positiv (FP)

sann negativ (TN)

Riktig og galt på ulike måter: feilmatriser



prediksjon = positiv
prediksjon = negativ

gull = positiv

sann positiv (TP)

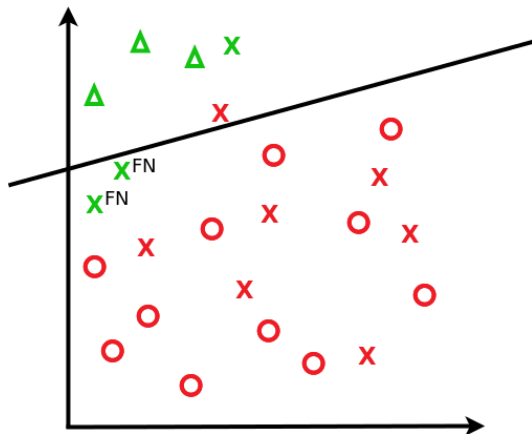
falsk negativ (FN)

gull = negativ

falsk positiv (FP)

sann negativ (TN)

Riktig og galt på ulike måter: feilmatriser



prediksjon = positiv
prediksjon = negativ

gull = positiv

sann positiv (TP)

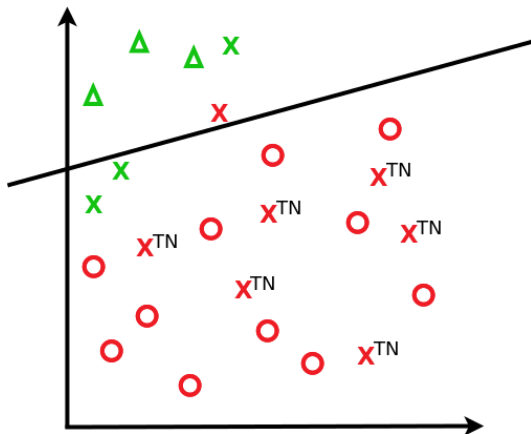
falsk negativ (FN)

gull = negativ

falsk positiv (FP)

sann negativ (TN)

Riktig og galt på ulike måter: feilmatriser



prediksjon = positiv
prediksjon = negativ

gull = positiv

sann positiv (TP)

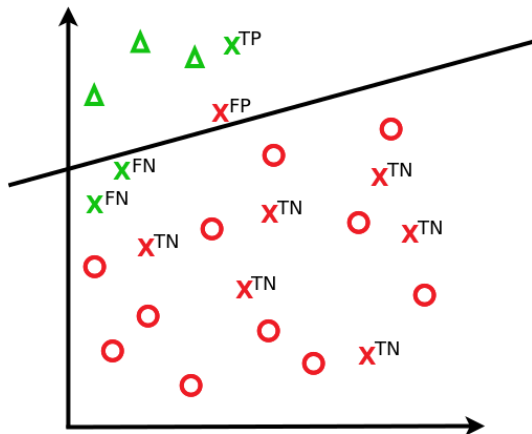
falsk negativ (FN)

gull = negativ

falsk positiv (FP)

sann negativ (TN)

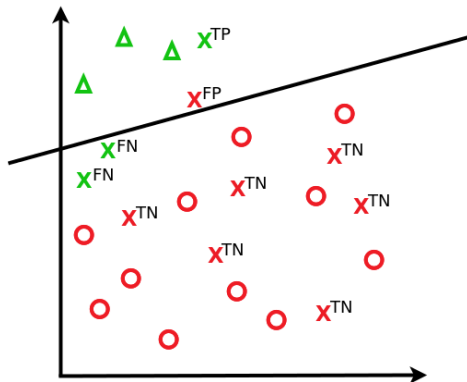
Riktig og galt på ulike måter: feilmatriser



prediksjon = positiv
prediksjon = negativ

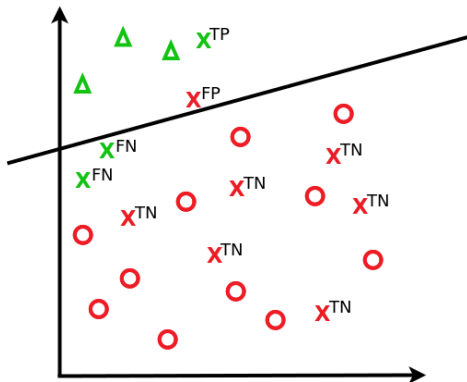
gull = positiv
sann positiv (TP)
falsk negativ (FN)

gull = negativ
falsk positiv (FP)
sann negativ (TN)



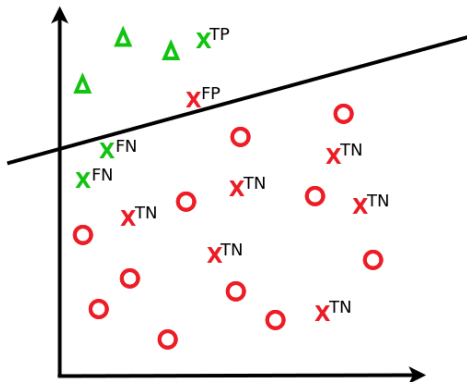
$$\text{Nøyaktighet} = \frac{TP+TN}{N}$$

	gull = positiv	gull = negativ
prediksjon = positiv	TP = 1	FP = 1
prediksjon = negativ	FN = 2	TN = 6



$$\text{Nøyaktighet} = \frac{TP+TN}{N}$$
$$= \frac{1+6}{10} = 0.7$$

	gull = positiv	gull = negativ
prediksjon = positiv	TP = 1	FP = 1
prediksjon = negativ	FN = 2	TN = 6

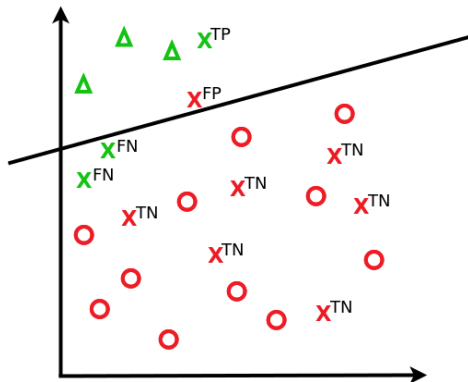


$$\text{Nøyaktighet} = \frac{TP+TN}{N}$$
$$= \frac{1+6}{10} = 0.7$$

$$\text{Presisjon} = \frac{TP}{TP+FP}$$

$$\text{Sensitivitet} = \frac{TP}{TP+FN}$$

	gull = positiv	gull = negativ
prediksjon = positiv	TP = 1	FP = 1
prediksjon = negativ	FN = 2	TN = 6

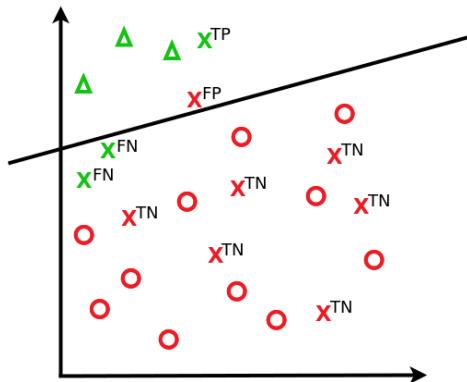


$$\text{Nøyaktighet} = \frac{TP+TN}{N}$$
$$= \frac{1+6}{10} = 0.7$$

$$\text{Presisjon} = \frac{TP}{TP+FP}$$
$$= \frac{1}{1+1} = 0.5$$

$$\text{Sensitivitet} = \frac{TP}{TP+FN}$$

	gull = positiv	gull = negativ
prediksjon = positiv	TP = 1	FP = 1
prediksjon = negativ	FN = 2	TN = 6

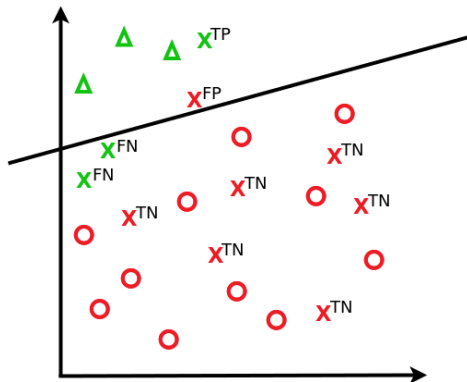


$$\text{Nøyaktighet} = \frac{TP+TN}{N}$$
$$= \frac{1+6}{10} = 0.7$$

$$\text{Presisjon} = \frac{TP}{TP+FP}$$
$$= \frac{1}{1+1} = 0.5$$

$$\text{Sensitivitet} = \frac{TP}{TP+FN}$$
$$= \frac{1}{1+2} = 0.33$$

	gull = positiv	gull = negativ
prediksjon = positiv	TP = 1	FP = 1
prediksjon = negativ	FN = 2	TN = 6



	gull = positiv	gull = negativ
prediksjon = positiv	TP = 1	FP = 1
prediksjon = negativ	FN = 2	TN = 6

$$\text{Nøyaktighet} = \frac{TP+TN}{N}$$
$$= \frac{1+6}{10} = 0.7$$

$$\text{Presisjon} = \frac{TP}{TP+FP}$$
$$= \frac{1}{1+1} = 0.5$$

$$\text{Sensitivitet} = \frac{TP}{TP+FN}$$
$$= \frac{1}{1+2} = 0.33$$

$$\text{F1-mål} = \frac{2 \times \text{presisjon} \times \text{sensitivitet}}{\text{presisjon} + \text{sensitivitet}}$$
$$= 0.4$$

- **Nøyaktighet** (*accuracy*) = $\frac{TP+TN}{N} = \frac{TP+TN}{TP+TN+FP+FN}$
 - Andelen korrekte prediksjoner.
 - Ikke egnet for ubalanserte klasser (dvs. stor forskjell i antall positive / negative eksempler).
- **Presisjon** (*precision*) = $\frac{TP}{TP+FP}$
 - Andelen identifiserte klassemedlemmer som var korrekte.
- **Sensitivitet** (*recall*) = $\frac{TP}{TP+FN}$
 - Andelen faktiske klassemedlemmer som ble identifisert.
 - Avveining: Positive prediksjoner for alle eksempler vil gi sensitivitet på 100%, men typisk svært dårlig presisjon.
- **F1-mål** (*F1-score*) = $\frac{2 \times \text{presisjon} \times \text{sensitivitet}}{\text{presisjon} + \text{sensitivitet}}$
 - Balanserer presisjon og sensitivitet i ett mål.
- **Manuell feilanalyse** også alltid anbefalt!

Mikrogjennomsnitt (*micro-average*)

- Summér først TP, FP, og FN for alle eksempler på tvers av alle klasser, og beregn så globale metrikker basert på disse.
- **Store** klasser vil få mest inflytelse på **mikro**gjennomsnittet.

Mikrogjennomsnitt (*micro-average*)

- Summér først TP, FP, og FN for alle eksempler på tvers av alle klasser, og beregn så globale metrikker basert på disse.
- **Store** klasser vil få mest innflytelse på **mikro**gjennomsnittet.

Makrogjennomsnitt (*macro-average*)

- Beregn først metrikkene for hver enkelt klasse, og ta så det globale snittet av disse.
- **Små** klasser vil få mye innflytelse på **makro**gjennomsnittet.

Mikrogjennomsnitt (*micro-average*)

- Summér først TP, FP, og FN for alle eksempler på tvers av alle klasser, og beregn så globale metrikker basert på disse.
 - **Store** klasser vil få mest innflytelse på **mikro**gjennomsnittet.
-

Makrogjennomsnitt (*macro-average*)

- Beregn først metrikkene for hver enkelt klasse, og ta så det globale snittet av disse.
 - **Små** klasser vil få mye innflytelse på **makro**gjennomsnittet.
-

Vektet

- Som makro, men tar høyde for ubalanse mellom klassene.
- Bidraget fra hver klasse vektes utfra antall (gull-)eksempler.

Intrinsisk

- Fokus på modellens ytelse på oppgaven den ble trent for.
- Evaluering mot data annotert med samme type informasjon som i treningsdatatene.
- Altså hva vi har antatt så langt.
- F.eks., for temaklassifikasjon; hva er F1-målet på et reservert testsett med dokumenter annotert med de samme tematiske klassene?

Intrinsisk

- Fokus på modellens ytelse på oppgaven den ble trent for.
- Evaluering mot data annotert med samme type informasjon som i treningsdatatene.
- Altså hva vi har antatt så langt.
- F.eks., for temaklassifikasjon; hva er F1-målet på et reservert testsett med dokumenter annotert med de samme tematiske klassene?

Ekstrinsisk

- Fokus på prediksjonenes effekt i en større eller 'nedstrøms' oppgave, 'ende-til-ende'.
- F.eks., for temaklassifikasjon; hvordan påvirkes brukertilfredsheten i et system for innholdsanbefalinger som bruker prediksjonene som et av flere trekk?



(Vi kommer tilbake til klassifikasjon flere ganger senere i semesteret!)

- Ikke-veiledet læring og **klyngeanalyse**
- k -means
- Hierarkisk klyngeanalyse
- Pensum:
- *Manning, Raghavan & Schütze*:
 - kapittel 16; introduksjonen, seksjon 16.2 og 16.4 frem til (men ikke med) 16.4.1
 - seksjon 17.1 og 17.6
- Shah: seksjon 7.1-7.3.